

DCIT 204/308 Joint DSA Semester

Project: Campus Dispatch & Optimization System

Project Context: University of Ghana (UG) Campus Dispatch & Taxi Optimization System (Targeting UG Hospital, verified campus taxis, student priority routing, and congestion management).

1. Project Work Allocation Matrix

Module Scope	Assigned Group	DSA Level Focus	Custom Data Structures & Algorithms Required
M1: Spec & M2: DB Loader	L200 (DSA I)	Linear / Database	Database schema (locations, roads, service_requests, resources, audit_events), CSV parser.
M3 (Linear DS) & M5 (FIFO Dispatch)	L200 (DSA I)	Linear Data Structures (DSA I)	Custom Dynamic Array, Stack (Audit/Undo), FIFO & Circular Queues.
M4: Searching & Sorting Engine	L200 (DSA I)	Basic Algorithms (DSA I)	Linear Search, Binary Search, Selection Sort, Insertion Sort, Merge Sort, Quick Sort.
M3 (Advanced Lists/Trees/Heaps) & M5 (Priority Dispatch)	L300 (DSA II)	Advanced Data Structures (DSA II)	Singly/Doubly Linked Lists, Skip Lists, Custom Max/Min Heap (Priority Queue), BST, Red-Black or B-Tree, Hash Table, Disjoint Set.

Module Scope	Assigned Group	DSA Level Focus	Custom Data Structures & Algorithms Required
M6: Tree & Hash Indexing Engine	L300 (DSA II)	Indexing Operations (DSA II)	Hash Table (with linked list collision chaining) for $O(1)$ request lookup; BST/B-Tree for indexing bookings by time and campus zone.
M7: Graph Route Engine	L300 (DSA II)	Graph Theory (DSA II)	Adjacency List (linked list graph nodes) & Matrix, BFS, DFS, Dijkstra (shortest path to UG Hospital), Prim/Kruskal.
M8: Optimisation Engine	L300 (DSA II)	Advanced Algorithm Design (DSA II)	Dynamic Programming (Knapsack allocation for taxi capacity/budget) + Greedy driver matching with failure counterexample.
M9: Testing & M10: Empirical Lab	Joint Effort (All)	Benchmarking & Verification	40+ Unit tests, 6 Trace tables, runtime execution experiments ($N=100$ to $50,000$) exporting to CSV and performance graphs.

2. Detailed Group Deliverables

L200 Group (DSA I Focus)

- **Database & Seed Data (M2):** Create schema.sql and seed data for 50 UG locations (UG Hospital, Night Market, Commonwealth, Pentagon, JSB, etc.), 100 connecting road

segments, and 30 verified campus taxis.

- **Linear Request Management (M3, M5):** Implement custom Array-backed lists and Circular Queues from scratch without using Java collections. Store standard student booking requests in FIFO order.
- **Audit & Undo Trail (M3):** Build a custom Stack implementation to manage request cancellations, driver status updates, and system rollback logs.
- **Sorting Engine (M4):** Build custom Merge Sort and Quick Sort algorithms to organize dispatch records by timestamp, distance, or verification status.

L300 Group (DSA II Focus)

- **Custom Linked Lists & Advanced List Structures (M3):** Build custom Singly/Doubly Linked Lists to support complex pointer management, Skip Lists, and collision resolution chains in Hash Tables.
- **Priority Matching Engine (M3, M5):** Build a custom Max/Min Heap (Priority Queue). Priority is calculated dynamically using team index-derived weights:
 - **Tier 1:** Disabled Students (highest priority score, immediate queue override).
 - **Tier 2:** Standard UG Students.
 - **Tier 3:** Non-student campus staff and external visitors.
- **Campus Routing Graph (M7):** Model campus locations and roads using Adjacency Lists (node linked lists). Use Dijkstra's algorithm to calculate optimal taxi routes to UG Hospital and BFS/DFS to verify route reachability during road blockages.
- **Congestion Heatmaps & Indexing (M6, M8):** Implement custom Hash Tables for instant booking lookups and Dynamic Programming (Knapsack algorithm) to optimize taxi placement in highly congested zones.
- **Greedy Allocation Counterexample (M8):** Document a formal counterexample demonstrating where a local greedy dispatch decision results in sub-optimal total wait times compared to dynamic programming.

3. Leader Execution Checklist & Milestones

1. **Week 1 - Architecture & Database:** L200 sets up SQL schema, dynamic arrays, and queues; L300 implements custom Linked Lists, interface contracts, heap structures, and graph models.
2. **Week 2 - Engine Development:** L200 finishes Search/Sort engine; L300 completes Dijkstra pathfinding (using Adjacency Lists), Priority Queue dispatching, and DP optimization routines.
3. **Week 3 - Testing & Evidence:** Conduct 40+ unit tests, generate 6 trace tables (Dijkstra, Quick Sort, Binary Search, Heap Operations), and handle boundary edge cases.
4. **Week 4 - Empirical Benchmarking & Defense:** Run execution timing tests ($N = 100$ to $50,000$), export CSV timing logs, produce performance graphs, and prepare every team member for individual oral defense on 1 data structure and 1 algorithm.

